Departamento de Ciência da Computação - DCC
Prof. Ricardo Martins
Site: https://ricardofm.net
Email: ricardo.martins@udesc.br
Ramal: 3481-7823
Sala: Bloco F – 2° piso (sala 20)



COMPILADORES

COM0002:
Ciência da Computação
5ª fase

EXEMPLO: MFCALC

Lembrete: formato do arquivo .y (*bison*)

```
%{
Prólogo: elementos precedentes
%}
Declarações
%%
Regras gramaticais
%%
Epílogo: elementos finais
```

PRÓLOGO: INCLUDES E PROTÓTIPOS

```
%{
#include <math.h>    /* Para as funções matemáticas... cos(), sin(), etc. */
#include "mfcalc.h" /* Contém a definição de 'symrec'.  */
int yylex (void);    /* Protótipo da função yylex */
void yyerror (char const *);  /* Protótipo da função yyerror */
%}
```

PRÓLOGO: MFCALC.H

```c
typedef double (*func_t) (double);

/* Tipo de dados para links/ponteiros de uma cadeia de símbolos...  */
struct symrec
{
  char *name;  /* Nome de um símbolo */
  int type;    /* Tipo de um símbolo: VAR ou FNCT */
  union
  {
    double var;       /* Valor de VAR */
    func_t fnctptr;  /* Valor de FNCT */
  } value;
  struct symrec *next;  /* Ponteiro para o próximo símbolo... */
};

typedef struct symrec symrec;

/* Aqui temos a tabela de símbolos: uma cadeia de 'struct symrec'...  */
extern symrec *sym_table;

symrec *putsym (char const *, int);
symrec *getsym (char const *);
```

DECLARAÇÕES

```
%union {
    double val;     /* Usado para o retorno de valores...  */
    symrec *tptr;   /* Usado p/ retorno de ponteiros (tabela de símbolos)  */
}
%token <val>  NUM           /* NÚMERO de dupla precisão  */
%token <tptr> VAR FNCT      /* Identificação de VARIÁVEL e FUNÇÃO  */
%type  <val>  exp

%right '='
%left '-' '+'
%left '*' '/'
%left NEG       /* Operador unário (-) */
%right '^'      /* Exponenciação */
%start input
```

REGRAS GRAMATICAIS

```
input:                  /* EPSILON */
      | input line
      ;
line: '\n'
      | exp '\n'           { printf ("\t%.10g\n", $1);           }
      | error '\n'         { yyerrok;                           }
      ;
exp:  NUM                  { $$ = $1;                           }
      | VAR                { $$ = $1->value.var;                }
      | VAR '=' exp        { $$ = $3; $1->value.var = $3;       }
      | FNCT '(' exp ')'   { $$ = (*($1->value.fnctptr))($3);   }
      | exp '+' exp        { $$ = $1 + $3;                      }
      | exp '-' exp        { $$ = $1 - $3;                      }
      | exp '*' exp        { $$ = $1 * $3;                      }
      | exp '/' exp        { $$ = $1 / $3;                      }
      | '-' exp  %prec NEG { $$ = -$2;                          }
      | exp '^' exp        { $$ = pow ($1, $3);                 }
      | '(' exp ')'        { $$ = $2;                           }
      ;
```

EPÍLOGO: FUNÇÕES *PUTSYM E *GETSYM

```
#include <stdio.h>
#include <ctype.h>
symrec *putsym (char const *sym_name, int sym_type) {
      symrec *ptr;
      ptr = (symrec *) malloc (sizeof (symrec));
      ptr->name = (char *) malloc (strlen (sym_name) + 1);
      strcpy (ptr->name,sym_name);
      ptr->type = sym_type;
      ptr->value.var = 0;     /* Define o valor igual a zero, mesmo para uma função...  */
      ptr->next = (struct symrec *)sym_table;
      sym_table = ptr;
      return ptr;
}
symrec *getsym (char const *sym_name) {
      symrec *ptr;
      for (ptr = sym_table; ptr != (symrec *) 0; ptr = (symrec *)ptr->next)
         if (strcmp (ptr->name,sym_name) == 0)
            return ptr;
      return 0;
}
```

EPÍLOGO: YYERROR E ARITH_FNCTS

```
/* Função que é chamada pelo yyparse, no caso de observar
a ocorrência de um erro sintático...  */
void yyerror (char const *s) {
     printf ("%s\n", s);
}
struct init {
     char const *fname;
     double (*fnct) (double);
};
struct init const arith_fncts[] =
{    /* Funções registradas... */
     "sin",  sin,
     "cos",  cos,
     "atan", atan,
     "ln",   log,
     "exp",  exp,
     "sqrt", sqrt,
     0, 0
};
```

EPÍLOGO: INIT_TABLE E MAIN

```
/* Aqui está a tabela de símbolos!!! Uma  cadeia da estrutura 'symrec'.  */
symrec *sym_table;
/* Inclui funções aritméticas na tabela... */

void init_table (void) {
     int i;
     symrec *ptr;
     for (i = 0; arith_fncts[i].fname != 0; i++) {
          ptr = putsym (arith_fncts[i].fname, FNCT);
          ptr->value.fnctptr = arith_fncts[i].fnct;
     }
}

int main (void) {
     init_table ();
     return yyparse ();
}
```

EPÍLOGO: YYLEX

```
int yylex (void) {
      int c;
      /* Ignora espaços em branco ou \t, e considera o primeiro caracter diferente disto... */
      while ((c = getchar ()) == ' ' || c == '\t');
      if (c == EOF)
          return 0;
      /* Verifica uma sequência de números... */
      if (c == '.' || isdigit (c)) {
          ungetc (c, stdin);
          scanf ("%lf", &yylval.val);
          return NUM;
      }
      /* Verifica o nome de um identificador... */
      if (isalpha (c)) {
          symrec *s;
          static char *symbuf = 0;
          static int length = 0;
          int i;
```

EPÍLOGO: YYLEX

```
        /* Define o comprimento inicial do buffer, igual a 40.  */
        if (length == 0)
             length = 40, symbuf = (char *)malloc (length + 1);
        i = 0;
        do {
             /* Se o buffer estiver cheio, faça-o ficar maior... */
                    if (i == length) {
                 length *= 2;
                 symbuf = (char *) realloc (symbuf, length + 1);
             }
             /* Adicione o caracter ao buffer... */
             symbuf[i++] = c;
             /* Obtenha outro caracter... */
             c = getchar ();
        } while (isalnum (c));
        ungetc (c, stdin);
        symbuf[i] = '\0';
        s = getsym (symbuf);
        if (s == 0)
             s = putsym (symbuf, VAR);
        yylval.tptr = s;
        return s->type;
    }
    /* Como o analisador léxico é definido como trivial,       */
    /* qualquer outro caracter é identificado como um token...*/
    return c;
}
```